ЛАБОРАТОРНА РОБОТА № 2

Тема: Детальний розгляд CSS: Flexbox, Grid, адаптивна верстка.
Мета: Набуття практичних навичок побудови сучасних макетів засобами Flexbox і CSS Grid, реалізації адаптивного та чуйного дизайну й організації стилів за методологією BEM.

Теоретичні відомості

У попередній роботі стилі застосовувалися здебільшого для оформлення окремих елементів. Ця робота присвячена іншому завданню – побудові макета сторінки, тобто взаємному розташуванню блоків та їх поведінці при зміні розмірів вікна. Сучасний CSS має для цього два спеціалізовані механізми: Flexbox для одновимірних розкладок і Grid для двовимірних.

2.1 Блокова модель і схеми позиціонування

Кожен елемент сторінки браузер представляє прямокутником, який складається з чотирьох вкладених областей: вміст (content), внутрішні відступи (padding), рамка (border) і зовнішні відступи (margin). Властивості `width` та `height` за замовчуванням задають розмір саме області вмісту, тому фактична ширина елемента виявляється більшою за оголошену на суму відступів і рамок. Це джерело більшості помилок у розрахунку макета.

Щоб `width` враховувала внутрішні відступи та рамку, застосовують альтернативну блокову модель. Її прийнято вмикати глобально на початку таблиці стилів:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Зовнішні відступи сусідніх блоків по вертикалі схлопуються (margin collapsing): замість суми двох відступів між блоками залишається більший із них. Це стосується лише вертикальних відступів звичайних блокових елементів і не діє всередині flex- та grid-контейнерів – ще одна причина віддавати перевагу цим механізмам у макетуванні.

Властивість `position` визначає схему позиціонування елемента:

- static – значення за замовчуванням, елемент розташовується у звичайному потоці документа;
- relative – елемент зміщується відносно свого місця в потоці, але місце за ним зберігається;
- absolute – елемент вилучається з потоку і позиціонується відносно найближчого предка з позиціонуванням, відмінним від static;
- fixed – позиціонування відносно вікна перегляду, елемент не прокручується разом зі сторінкою;
- sticky – гібрид: елемент поводиться як relative, доки не досягне заданої межі, після чого «прилипає» до неї.

Порядок накладання елементів, вилучених з потоку, регулюється властивістю `z-index`, яка діє лише на позиціоновані елементи в межах спільного контексту накладання.

2.2 Каскад, специфічність і спадкування

Коли на один елемент претендує кілька правил, застосовується те, чиє походження та вага вищі. Спершу враховується походження таблиці стилів, потім специфічність селектора, і лише за їх рівності – порядок оголошення: перемагає правило, розташоване нижче.

Специфічність зручно уявляти як трійку чисел (a, b, c): a – кількість ідентифікаторів, b – класів, атрибутів і псевдокласів, c – типів елементів і псевдоелементів. Селектор `#menu .item a` має специфічність (1, 1, 1) і переважає над `.nav .list .item a` зі специфічністю (0, 3, 1), попри більшу кількість складових. Вбудований стиль в атрибуті `style` переважає над будь-

яким селектором, а прапорець `!important` – над усім, тому в навчальних і виробничих проєктах його застосування зводять до мінімуму.

Частина властивостей (шрифт, колір тексту, міжрядковий інтервал) успадковується нащадками, частина (відступи, рамки, розміри) – ні. Ключові слова `inherit`, `initial`, `unset` та `revert` дають змогу керувати цим явно.

2.3 Одиниці вимірювання та змінні

Абсолютні одиниці (px, pt, cm) не залежать від контексту, відносні – залежать. Найчастіше застосовують: `em` – відносно розміру шрифту самого елемента, `rem` – відносно розміру шрифту кореневого елемента, `%` – відносно відповідного розміру батьківського блока, `vw` і `vh` – відносно ширини та висоти вікна перегляду, `fr` – частка вільного простору у grid-розкладці. Використання `rem` для розмірів шрифту та відступів робить макет масштабованим: зміна `font-size` кореневого елемента пропорційно змінює весь макет.

Власні властивості (CSS-змінні) оголошуються з подвійним дефісом і читаються функцією `var()`. Вони успадковуються, тому їх зручно визначати в `:root` і перевизначати в окремих секціях або темах:

```
:root {
  --gap: 1rem;
  --color-accent: #2f6fed;
  --radius: 8px;
}

.card {
  padding: var(--gap);
  border-radius: var(--radius);
  border: 1px solid var(--color-accent);
}
```

Корисними є також функції `calc()`, `min()`, `max()` і `clamp()`. Останню застосовують для плавної типографіки: `font-size: clamp(1rem, 0.5rem + 2vw, 2rem);` задає розмір, що зростає разом із шириною вікна, але не виходить за вказані межі.

2.4 Flexbox: одновимірні розкладки

Flexbox керує розташуванням елементів уздовж однієї осі. Контейнер оголошується властивістю `display: flex`, після чого його безпосередні нащадки стають flex-елементами і втрачають власну блокову поведінку. Головна вісь задається властивістю `flex-direction` (row, row-reverse, column, column-reverse), поперечна їй перпендикулярна.

Основні властивості контейнера:

- justify-content – розподіл елементів уздовж головної осі (flex-start, center, space-between, space-around, space-evenly);
- align-items – вирівнювання елементів поперечної осі (stretch, flex-start, center, baseline);
- flex-wrap – дозвіл переносити елементи на новий рядок (nowrap, wrap);
- align-content – розподіл рядків, коли їх декілька;
- gap – відстань між елементами, що не створює зайвих відступів по краях.

Поведінку окремого елемента визначає скорочення `flex`, яке об'єднує три властивості: `flex-grow` (частка вільного простору, яку елемент забирає), `flex-shrink` (наскільки він стискається за браку місця) та `flex-basis` (початковий розмір). Значення `flex: 1 1 0` робить елементи однаковими за шириною незалежно від вмісту, а `flex: 0 0 auto` фіксує розмір за вмістом.

```
.toolbar {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.toolbar__title { flex: 1 1 auto; }   /* займає весь вільний простір */
.toolbar__actions { flex: 0 0 auto; } /* зберігає розмір за вмістом */
```

Властивість `order` змінює візуальний порядок елементів, не змінюючи порядку в розмітці. Зловживати нею не варто: порядок читання з клавіатури та програмою екранного доступу залишається таким, як у HTML, що погіршує доступність.

2.5 CSS Grid: двовимірні розкладки

Grid оперує рядками і колонками одночасно, тому підходить для каркаса сторінки. Контейнер оголошується як `display: grid`, а сітка описується властивостями `grid-template-columns` і `grid-template-rows`. Одиниця `fr` розподіляє вільний простір, а функція `repeat()` скорочує запис повторюваних доріжок.

```
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-tem

plate-areas:
    "header header"
    "aside  main"
    "footer footer";
  min-height: 100vh;
  gap: 1rem;
}

.layout__header { grid-area: header; }
.layout__aside  { grid-area: aside; }
.layout__main   { grid-area: main; }
.layout__footer { grid-area: footer; }
```

Іменовані області (`grid-template-areas`) роблять макет наочним просто з тексту стилів. Альтернативно елемент розміщують за номерами ліній сітки: `grid-column: 1 / 3;` або `grid-row: span 2;`.

Для галерей і карткових списків застосовують автоматичне заповнення, яке дає адаптивність без жодного медіа-запиту:

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 1rem;
}
```

Ключове слово `auto-fill` створює стільки колонок, скільки вміщується, а функція `minmax()` задає їх мінімальну та максимальну ширину. Варіант `auto-fit` відрізняється тим, що згортає порожні доріжки, розтягуючи наявні елементи на всю ширину.

Вирівнювання в Grid виконують ті самі властивості, що й у Flexbox, але окремо для кожної осі: `justify-items` та `align-items` – для елементів усередині їхніх комірок, `justify-content` та `align-content` – для сітки в межах контейнера.

Практичне правило вибору: Grid – для каркаса сторінки та будь-якої розкладки, де рядки й колонки мають узгоджуватися між собою; Flexbox – для панелей, груп кнопок, елементів навігації та інших випадків, коли елементи вибудовуються в один ряд або стовпець.

2.6 Адаптивний і чуйний дизайн

Чуйний дизайн (responsive design) передбачає, що макет плавно пристосовується до розміру вікна перегляду. Обов’язковою умовою є метатег вікна перегляду в розділі head, без якого мобільний браузер масштабує сторінку як настільну:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Точки перелому оголошують медіа-запитами. Сучасний підхід – mobile first: базові стилі описують найвужчий екран, а медіа-запити з умовою `min-width` послідовно ускладнюють макет для ширших екранів.

```css
.layout {
  display: grid;
  grid-template-columns: 1fr;      /* мобільна версія: один стовпець */
  gap: 1rem;
}

@media (min-width: 48em) {         /* від 768 px */
  .layout { grid-template-columns: 200px 1fr; }
}

@media (min-width: 75em) {         /* від 1200 px */
  .layout { grid-template-columns: 240px 1fr 280px; }
}
```

Точки перелому доцільно призначати за поведінкою макета, а не за моделями пристроїв: перелом ставлять там, де верстка починає виглядати гірше. Крім ширини, медіа-запити перевіряють орієнтацію (`orientation`), можливості пристрою введення (`hover`, `pointer`) та побажання користувача – наприклад, `prefers-color-scheme` для темної теми і `prefers-reduced-motion` для зменшення анімації.

Зображення в чуйному макеті масштабують правилом `max-width: 100%; height: auto;`, а для різних щільностей екрана застосовують атрибути `srcset` і `sizes` або елемент `<picture>`.

2.7 Переходи, анімації та трансформації

Властивість `transition` задає плавну зміну значень при зміні стану елемента, наприклад при наведенні курсора. Анімувати варто лише `transform` та `opacity` – саме ці властивості браузер обробляє без перерахунку геометрії сторінки, тому анімація залишається плавною.

```css
.card {
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover,
.card:focus-visible {
  transform: translateY(-4px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.15);
}

@media (prefers-reduced-motion: reduce) {
  .card { transition: none; }
}
```

Складніший рух описують правилом `@keyframes` і властивістю `animation`, яка задає тривалість, функцію згладжування, затримку, кількість повторів і напрямок відтворення.

2.8 Методологія BEM

Зі зростанням проєкту стилі стають некерованими: селектори перетинаються, специфічність зростає, а зміна одного правила ламає інші сторінки. BEM (Block – Element – Modifier) пропонує іменування, що усуває цю проблему.

- Блок – самостійний компонент, який має сенс сам собою: card, menu, form;
- Елемент – частина блока, що не існує окремо від нього; записується через подвійне підкреслення: card__title, menu__item;
- Модифікатор – стан або різновид блока чи елемента; записується через подвійний дефіс: card--featured, menu__item--active.

```html
<article class="card card--featured">
  <h2 class="card__title">Назва</h2>
  <p class="card__text">Опис</p>
  <a class="card__link" href="#">Докладніше</a>
</article>
```

```css
.card { display: grid; gap: 0.5rem; padding: var(--gap); }
.card__title { margin: 0; font-size: 1.25rem; }
.card--featured { border-color: var(--color-accent); }
```

Правила BEM прості: селектори складаються з одного класу, вкладеність не застосовується, ідентифікатори для оформлення не використовуються. Завдяки цьому специфічність усіх правил однакова, а порядок їх застосування стає передбачуваним.

2.9 Доступність макета

Макет має залишатися придатним до використання з клавіатури та програмами екранного доступу. Не слід прибирати обвідку фокуса – її замінюють власним оформленням через псевдоклас `:focus-visible`. Порядок елементів у розмітці має відповідати логічному порядку читання, а контрастність тексту і тла – становити щонайменше 4,5:1 для звичайного тексту. Ці показники зручно перевіряти вкладкою Lighthouse інструментів розробника браузера.

Порядок виконання роботи

1. Розробіть макет сторінки заданої структури: шапка, головна навігація, бічна панель, основна область вмісту та підвал. Каркас сторінки реалізуйте засобами CSS Grid з використанням іменованих областей (`grid-template-areas`).
   - шапка й підвал займають усю ширину сторінки;
   - бічна панель має фіксовану ширину, основна область – займає решту простору;
   - підвал притиснутий до низу вікна навіть за малого обсягу вмісту.
2. Наповніть основну область карточним списком щонайменше з восьми карток. Кожна картка містить зображення, заголовок, короткий опис та посилання. Список має бути адаптивним без медіа-запитів – за рахунок `repeat(auto-fill, minmax(...))`.
3. Побудуйте головну навігацію та панель дій у шапці засобами Flexbox: логотип ліворуч, меню по центру, кнопки праворуч, вертикальне вирівнювання по центру, відступи через `gap`.
4. Реалізуйте чуйну поведінку макета за підходом mobile first щонайменше з двома точками перелому:
   - до 768 px – усі області одна під одною, навігація згортається у вертикальний список;
   - від 768 px – з’являється бічна панель;
   - від 1200 px – макет набуває остаточного вигляду, ширина вмісту обмежується.
5. Опишіть у `:root` набір власних властивостей (кольори, базовий відступ, радіус заокруглення, типографічні розміри) і використайте їх у всіх правилах замість літеральних значень. Реалізуйте на їх основі темну тему через `prefers-color-scheme`.
6. Додайте плавні переходи для карток і елементів навігації при наведенні та фокусуванні. Анімуйте лише `transform` та `opacity`; передбачте вимкнення анімації правилом `prefers-reduced-motion`.
7. Усі стилі організуйте за методологією BEM: один клас на селектор, без вкладеності та ідентифікаторів. Стилі винесіть в окремий файл і підключіть посиланням.
8. Перевірте роботу засобами інструментів розробника браузера:
   - режим адаптивності – на ширинах 360, 768, 1024 та 1440 px;
   - панель Layout – відображення сітки Grid та ліній Flexbox;
   - вкладка Lighthouse – показники розділів Accessibility та Best Practices не нижче 90.
9. Вихідні коди вивантажте у git-репозиторій окремими змістовними комітами.
10. Результати роботи оформіть звітом.

Варіанти завдання

Предметна область обирається за таблицею; структура макета та вимоги до верстки однакові для всіх варіантів.

Варіант	Зміст завдання
1	Каталог товарів інтернет-магазину: фільтри у бічній панелі, картки товарів у сітці.
2	Новинний портал: рубрики у бічній панелі, картки публікацій із зображеннями анонсів.
3	Портфоліо розробника: навігація розділами, сітка проєктів із коротким описом.
4	Панель керування сервісом: меню розділів, картки показників і зведених даних.

Зміст звіту

1. Назва й мета лабораторної роботи.
2. Схема макета сторінки з позначенням областей сітки та точок перелому.
3. Лістинги ключових фрагментів HTML і CSS із поясненнями.
4. Скріншоти результатів роботи на трьох ширинах вікна перегляду.
5. Скріншот панелі Layout інструментів розробника з увімкненим відображенням сітки.
6. Скріншот звіту Lighthouse із показниками Accessibility та Best Practices.
7. Знімок екрана з файловою структурою проєкту та посилання на git-репозиторій.
8. Висновки по лабораторній роботі.

Контрольні питання

1. Чим відрізняються значення content-box і border-box властивості box-sizing?
2. Як обчислюється специфічність селектора та в якому випадку вона не має значення?
3. У чому різниця між одиницями em, rem, % і fr?
4. Що задають властивості flex-grow, flex-shrink і flex-basis?
5. Коли доцільніше застосувати Grid, а коли Flexbox?
6. Чим відрізняються ключові слова auto-fill та auto-fit у функції repeat()?
7. Що таке підхід mobile first і чим він кращий за протилежний?
8. Чому анімувати рекомендується саме transform та opacity?
9. Як записуються блок, елемент і модифікатор у методології BEM?
10. Які вимоги доступності слід ураховувати під час верстки макета?

Корисні посилання

- Специфікації та документація:
- https://developer.mozilla.org/uk/docs/Web/CSS
- https://www.w3.org/TR/css-flexbox-1/
- https://www.w3.org/TR/css-grid-1/
- Довідники з Flexbox і Grid:
- https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- https://css-tricks.com/snippets/css/complete-guide-grid/
- https://gridbyexample.com/
- Інтерактивні тренажери:
- https://flexboxfroggy.com/
- https://cssgridgarden.com/
- Методологія BEM:
- https://en.bem.info/methodology/
- https://getbem.com/naming/
- Адаптивність і доступність:
- https://web.dev/learn/design/
- https://www.w3.org/WAI/WCAG22/quickref/
- https://developer.chrome.com/docs/lighthouse/overview
- Підтримка властивостей браузерами – https://caniuse.com/